

Document Trap

Your season-end basketball tournament starts next week, so you want to brush up on the rules. You upload your league’s 200–page rulebook and ask: “How many fouls until I’m out of the game?”

ChatGPT answers: “Five fouls and you foul out.”

Wait a second. In last year’s tournament, you remember a player picking up five fouls and staying in the game. So you dig through the rulebook yourself. Regular season: five fouls, exactly what ChatGPT said. But near the end there’s a special section for tournaments, and in those games players get six.

The AI pulled the standard limit and missed the exception. The answer wasn’t made up. It was incomplete. **The trap is thinking ‘uploaded’ means ‘fully read.’**

IT’S ABOUT THE CONTEXT WINDOW

As you learned earlier in the course, the model reads its full context window every time you chat. Your two-hundred-page rulebook comes out around 100,000 tokens. That’s more than some models will load at once, and even when it fits, the system still needs room for your conversation and its answer.

So when a long document doesn’t fit, and many don’t, the system runs a process on it instead:

- 1 The document gets split into chunks, each a paragraph or two long.**
- 2 Each chunk gets a meaning vector. Same idea you saw in the Embeddings lesson.**
- 3 Your question becomes a vector too. The closest chunks get loaded.**

Now you know what happened with the rulebook. It became a few hundred chunks. Your question pulled the chunks closest to it in meaning, and “how many fouls until I’m out of the game?” sits right next to the regular-season foul rules. The tournament section near the back is about tournaments first and fouls second, so it didn’t make the cut. The model answered from the chunks it got: five fouls.

RETRIEVAL

There’s a name for what just happened: **retrieval**. When the system can’t load everything, it retrieves the pieces that match your question. Done well, this finds you a specific answer in a 200–page rulebook in seconds. Done poorly, the wrong pieces get pulled, and the model answers from incomplete evidence.

WHAT YOU CAN DO

You can’t always tell what got retrieved. But you can steer it. Five moves help.

The five moves

- 1 Point to the section by name**

Include keywords from the document in your question. The system uses them to find matching chunks.

VAGUE

What are the rules on flagrant fouls?

POINTED

What does the technical fouls chapter say about flagrant fouls?
- 2 Ask one question at a time**

Multi-part questions force the system to retrieve different chunks for different parts. Break it up. One focused question per turn gives retrieval the best shot.

COMBO QUESTION

Summarize the policy, list the exceptions, and tell me what to do.

ONE THING AT A TIME

What does the policy say about late assignments? Quote the line.
- 3 Paste the section directly into your prompt**

If you already know the paragraphs that matter, paste them into your message. That puts them in context with no retrieval lottery.

RETRIEVAL LOTTERY

Does my technical fouls clause apply if both players are involved?

PASTED IN DIRECTLY

Here’s the clause: ‘A technical foul is assessed when...’ Does this apply if both players are involved?
- 4 Upload only what’s relevant**

Instead of the whole rulebook, upload just the chapter you need. Fewer chunks competing for retrieval slots means the right ones are more likely to make the cut.

WHOLE RULEBOOK

[Uploads the entire 200–page rulebook] What does it say about flagrant fouls?

JUST THE CHAPTER

[Uploads only the 3–page fouls chapter] What does it say about flagrant fouls?
- 5 Ask the AI to quote**

Add ‘quote the exact section you’re basing this on’ to your question. If the quote is missing or doesn’t match the document, retrieval probably failed.

NO RECEIPTS

What does the rulebook say about flagrant fouls?

ASK FOR THE QUOTE

What does the rulebook say about flagrant fouls? Quote the exact line you’re basing it on.

All five share one idea: make the right chunks easy to find. The more specific your question, the more reliable the answer.

It answers from what it retrieved.

Not from having read the whole thing.